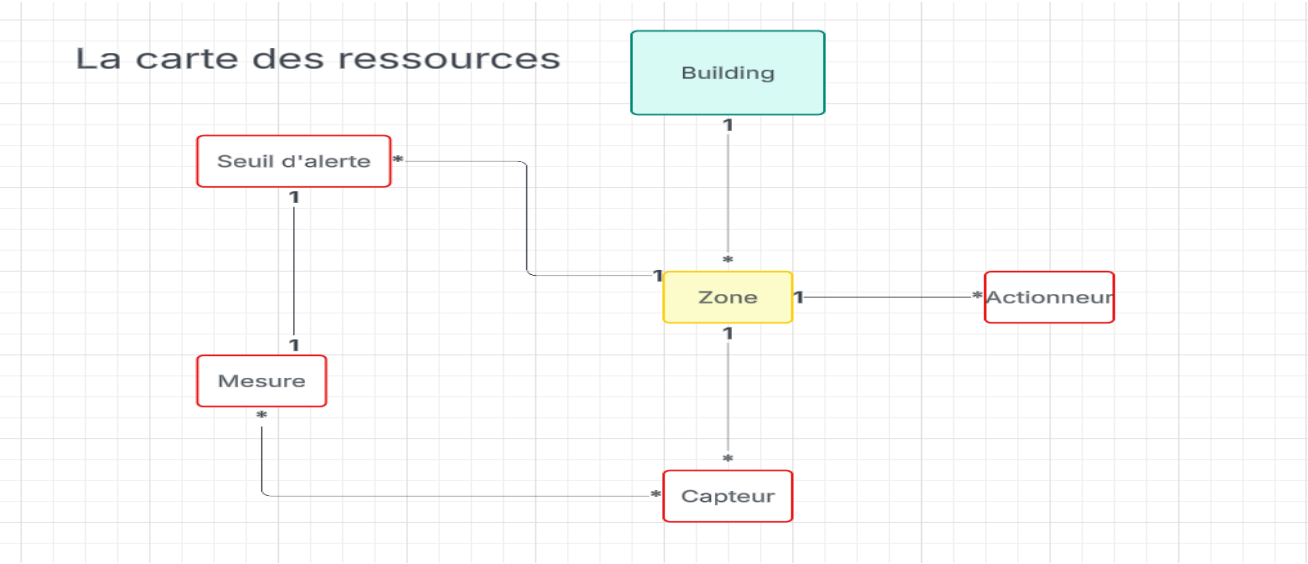
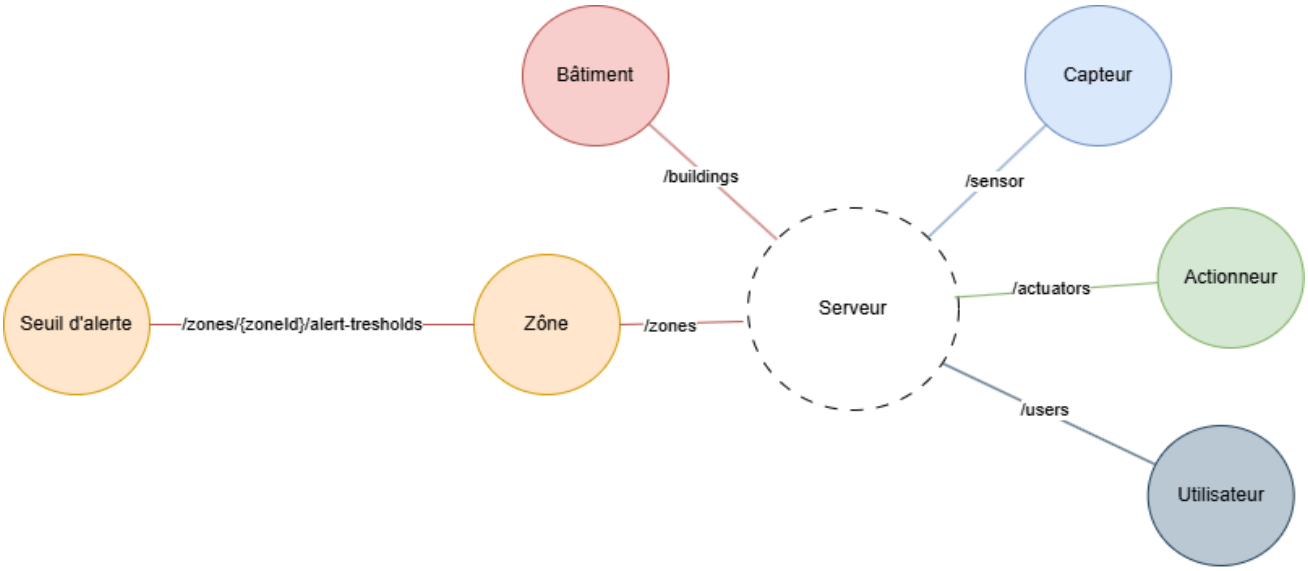
1. Carte des ressources et URIs



Hiérarchie des ressources




```
Building
├── Zone
│   ├── Sensor
│   │   └── Measurement
│   ├── Actuator
│   └── Alert Threshold
└── user
```

3. Au minimum 5 décisions de design argumentées

- Des urls simple et concise. Pour la lisibilité
- On a pris la décision de séparer les urls “bâtiments” et “zones” pour les traiter indépendamment
- Seuils d'alerte comme sous-ressource de la zone pour identifier rapidement
- Des querry parameters seront utiliser pour la pagination et les filtres de recherche.
- Pour le endpoint des buildings, le token utilisateur sera passer dans le header des requêtes (pour toujours identifier ce qu’il a l’autorisation de voir, modifier, créer etc...)

endpoints

- /buildings
- /zones
- /actuators
- /users
- /sensors
- /alert-threshold

4. Matrice contraintes x endpoints

Endpoint	C1 (Réseau)	C2 (Volumétrie)	C3 (Fiabilité)	C4 (Concurrence)
GET				
/buildings	Non	Oui	non	non
/buildings/{buildingId}	Non	non	Non	non
/buildings/{buildingId}/zones	non	Oui	Non	non
/zones/{zoneId}	non	non	Non	non
/zones/{zoneId}/sensors	non	oui	Non	non
/sensors/{sensorId}/measurements	non	oui	Non	non
/sensors/{sensorId}	non	non	Non	non
/zones/{zoneId}/actuators	non	oui	Non	non
/actuators/{actuatorId}	non	non	Non	non
/zones/{zoneId}/alert-thresholds	non	non	Non	non
/alert-thresholds/{thresholdId}	non	non	Non	
/zones/{zoneId}/alerts	non	non		
POST				
/buildings	Oui Gerer les doublons	Non	Oui Gerer les doublons	Oui Gerer les doublons
/zones	Oui Gerer les doublons	Non	Oui Gerer les doublons	Oui Gerer les doublons
/sensors	Oui Gerer les doublons	Non	Oui Gerer les doublons	Oui Gerer les doublons

/actuators	Oui Gerer les doublons	Non	Oui Gerer les doublons	Oui Gerer les doublons
/alert-thresholds	Oui Gerer les doublons	Non	Oui Gerer les doublons	Oui Gerer les doublons
PUT				
/buildings/{buildingsId}	Non	Non	Non	Oui. Si modification simultanée alors dernière modification renvoie une erreur
/zones/{zoneId}	Non	Non	Non	Oui. Si modification simultanée alors dernière modification renvoie une erreur
/sensors/{sensorId}	Non	Non	Non	Oui. Si modification simultanée alors dernière modification renvoie une erreur
/actuators/{actuatorId}	Non	Non	Non	Oui. Si modification simultanée alors dernière modification renvoie une erreur
/alert-thresholds/{thresholdId}	Non	Non	Non	Oui. Si modification simultanée

				alors dernière modification renvoie une erreur
DELETE				
/buildings/{buildingsId}	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée	Non	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée	Oui. Si suppresion simultanée alors retour disant que la ressource est déjà supprimée
/zones/{zoneId}	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée	Non	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée
/sensors/{sensorId}	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée	Non	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée
/actuators/{actuatorId}	Oui. Si suppression simultané	Non	Oui. Si suppression simultané	Oui. Si suppression simultanée alors retour

	e alors retour disant que la ressource est déjà supprimée		e alors retour disant que la ressource est déjà supprimée	disant que la ressource est déjà supprimée
/alert-thresholds/{thresholdId}	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée	Non	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée	Oui. Si suppression simultanée alors retour disant que la ressource est déjà supprimée

```
## Code de statut
GET réussi --> 200 avec header contenant les infos
GET échoué --> 204 si pas de contenu ou 400 si erreur
POST réussi --> 201 avec ressource créé
POST échoué --> 400 avec erreur ou 425 si duplicata possible
PUT réussi --> 200 avec ressource modifiée
PUT échoué --> 400 avec erreur
DELETE réussi --> 200
DELETE échoué --> 400 avec erreur
```

Carte des relations

5. Analyse de 2 scénarios critiques

Scenario A — Commande dupliquée (réseau instable)

Contexte : L'opérateur appuie sur "Activer la ventilation zone 3". Le réseau est instable, la requête part deux fois.

Prérequis : Le client génère un UUID unique (`Idempotency-Key`) avant d'envoyer la commande. Ce même UUID est utilisé pour les deux requêtes.

Séquence HTTP

Requête 1 (arrive en premier) :

POST /actuators/42/commands

Réponse 1 :

HTTP/1.1 201 Created

Location: /actuators/42/commands/cmd-789

-> L'actionneur est activé. La clé d'idempotence est stockée côté serveur.

Requête 2 (doublon, arrive quelques secondes après) :

POST /actuators/42/commands

Réponse 2 :

HTTP/1.1 409 Conflict

-> Le serveur reconnaît le doublon via l'`Idempotency-Key`. L'actionneur n'est pas déclenché une seconde fois. Le client peut suivre le lien vers la commande existante pour vérifier son statut.

Note alternative : Certaines implémentations préfèrent retourner `200 OK` avec la réponse originale (au lieu de `409`). Le choix du `409` rend le doublon explicite pour le client, ce qui est plus informatif pour l'opérateur mobile.

En cas d'échec réseau total (aucune réponse reçue) :

Le client peut retransmettre avec la même `Idempotency-Key` en toute sécurité. Si la première requête avait été traitée, il recevra le `409`. Si elle ne l'avait pas été, il recevra le `201`.

Scénario B — Conflit d'opérateurs (Alice et Bob)

Contexte : Alice veut mettre le chauffage zone 5 en puissance 3. Au même moment, Bob veut le mettre en puissance 1. Le chauffage est actuellement en puissance 2.

Prérequis : Les deux opérateurs ont préalablement fait un `GET /actuators/99` et ont reçu le même ETag.

Séquence HTTP

Étape 1 — Alice et Bob lisent l'état actuel :

GET /actuators/99

HTTP/1.1 200 OK

ETag: "v5"

Les deux reçoivent `ETag: "v5"`.

Étape 2 — Alice envoie sa commande en premier :

POST /actuators/99/commands

Réponse Alice :

HTTP/1.1 201 Created

-> La commande est acceptée. L'actionneur passe en puissance 3. Son ETag devient `"v6"`.

Étape 3 — Bob envoie sa commande (arrive juste après) :

POST /actuators/99/commands

Réponse Bob :

HTTP/1.1 409 Conflict

-> Bob est informé que l'état a changé. **Personne ne "gagne" silencieusement** : Alice a réussi car elle était la première, et Bob est explicitement notifié du conflit.

Étape 4 — Bob rafraîchit et retente (s'il le souhaite) :

GET /actuators/99

HTTP/1.1 200 OK

ETag: "v6"

Bob voit que la puissance est maintenant à 3 (modifiée par Alice). Il peut décider :

- De renvoyer sa commande avec `If-Match: "v6"` pour forcer la puissance à 1

- Ou de ne rien faire, estimant que la puissance 3 d'Alice est correcte

Résumé du scénario :

Opérateur	Requête	Résultat	Pourquoi
Alice	`POST` avec `If-Match: "v5"`	`201 Created`	ETag correspond, commande acceptée
Bob	`POST` avec `If-Match: "v5"`	`409 Conflict`	ETag obsolète, état modifié par Alice
Bob (retry)	`POST` avec `If-Match: "v6"`	`201 Created`	ETag correspond, commande acceptée si Bob insiste

Ce mécanisme garantit qu'aucun changement n'est écrasé silencieusement (contrainte C4), tout en laissant l'opérateur décider de la marche à suivre.

Compte rendu :

Points forts :

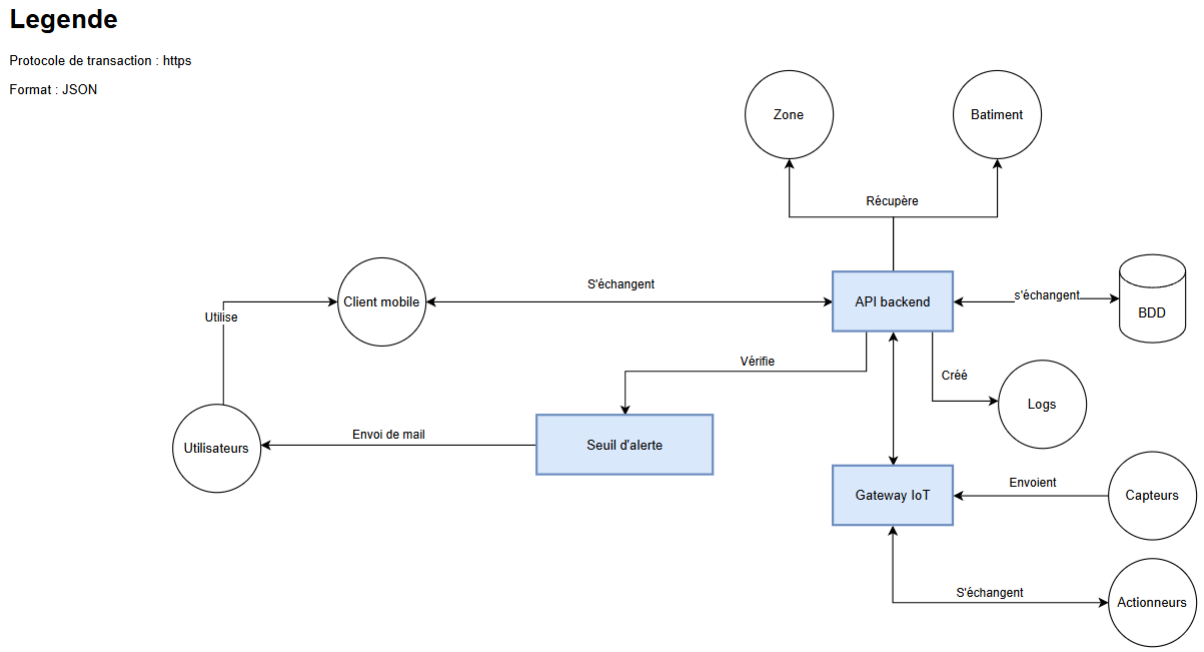
- Codes erreurs normalisés via Errors avec code, message, details
- Enums bien pensés

Points faibles :

- Post building : il reste les coordonnées du truc des voitures
- Dans la requête post les éléments sont mis dans les paramètres et non dans le body
- Dans les schémas les paramètres sont en UUID
- Manque a developper les endpoints de measurements
- Oubliés de retirer le INVALID_COORDINATES du schéma error

Threat model

1- DFD



2 — Endpoints sélectionnés pour l'analyse

#	Enpoint (méthode + path)	Données manipulées	Raison de la sélection
1	GET /sensors/{sensorId}/measurements	Toutes les mesures d'un capteur	Beaucoup de données récupérer ce qui va avoir un impact si pas bien traiter
2	PUT /actuators/{actuatorId}	Possibilité de pouvoir modifier l'état d'un actionnaire	Si pas bien protéger n'importe qui pourra actionner/désactiver un dispositif pilotable
3	PUT /alert-thresholds/{thresholdId}	Possibilité de pouvoir modifier le seuil d'alerte	Pour éviter que tout le monde puisse changer la valeur pour un seuil d'alerte

3 – Tableau des menaces

#	Endpoint	Menace identifée	Catégorie OWASP	Vraisemblance (F / M / É)	Impact (F / M / É)	Risque résulta	Contremesure proposée	Compromis identifié

			API Top 10			nt (F / M / É / C)		
1	1	Trop d'élément dans la réponse (fuite de donnée inutile et compromettante)	API3 (2019) Excessive Data Exposure	M	É	C	Vérifier les données renvoyer à l'utilisateur	Des informations en moins
1 *	1	Beaucoup de valeur en bdd	API3 (2023)	É	É	C	Ajouter une pagination	Faire un compromis sur le choix de pagination
2	Tous	Broken Access control	A01 (2025)	É	É	É	Bloquer les Access par défaut Créer des rôles	Avoir des rôles pour pouvoir modifier les données en base, Avoir une Gateway
3	3	Un utilisateur non autorisé peut modifier un seuil d'alerte en manipulant l'ID dans l'URL	API5 (BFLA)	É	É	C	Mettre en place un controle d'autorisation coté back pour vérif que seul un user avec le bon role peut modifier un seuil, et vérifier qu'il possède les droits sur la ressource ciblée	Augmentation de la complexité back et legere latence due aux vérif d'autorisation
3 *	3	Modif de seuil avec des valeurs invalides ou extremes	API3	M	É	É	Validation stricte des valeurs coté back (types, limites ..) et biensur implémenter les règles métier pour empêcher les valeurs dangereuses	Peut limiter la flexibilité pour certains cas d'usage
4	Tous	Security misconfiguration	A02 (2025)	M	É	É	Bloquer les ports non utilisés, ne pas avoir de compte par defaut,	

							Avoir une bonne gestion d'erreur	
5	2	Injection	A05 (2025)	M	É	C	Controller l'arrivée des données pour qu'elles soient correctes et bien formattées	Rajouter de la logique de controle des données

4 — Priorisation des risques

4–1 Risque Prioritaire n°1

Élément	Réponse
Menace (réf. tableau §3)	Injection
Endpoint concerné	PUT /actuators/{actuatorId} ou PUT /alert-thresholds/{thresholdId}
Pourquoi ce risque est prioritaire	Si une injection réussie, l'attaquant gagne l'accès potentiel a toute la base de données
Conséquence métier si exploité	Accès compromis aux données et perte de confiance sur la donnée
Contre-mesure retenue	Controller l'arrivée des données pour qu'elles soient correctes et bien formattées
Compromis accepté	Rajouter de la logique de contrôle des données

4–2 Risque Prioritaire n°2

Élément	Réponse
Menace (réf. tableau §3)	Broken access control
Endpoint concerné	GET /sensors/{sensorId}/measurements
Pourquoi ce risque est prioritaire	Que n'importe qui puisse avoir assez à des informations alors qu'il n'a pas les droits pour voir ces infos
Conséquence métier si exploité	Récupérer des informations sensibles et potentiellement compromettre la base de données
Contre-mesure retenue	Bloquer les Access par défaut Et créer des rôles

Compromis accepté	Avoir des rôles pour pouvoir modifier les données en base, Avoir une Gateway

4–3 Risque Prioritaire n°3

Élément	Réponse
Menace (réf. tableau §3)	Trop d'éléments dans la réponse
Endpoint concerné	GET /sensors/{sensorId}/measurements
Pourquoi ce risque est prioritaire	Il peut retourner un grand volume de données… Sans limitation ça peut surcharger le serveur/bdd
Conséquence métier si exploité	Le système pourrait devenir lent ou indispo si trop de données sont demandées en même temps
Contre-mesure retenue	Une pagination et limiter le nombre de résultats retournés par requête
Compromis accepté	L'utilisateur doit effectuer plusieurs requêtes pour récup toutes les données

5- Hypothèse et limites

Hypothèses :

1. Il faut renforcer les contrôles d'accès des Endpoint

2. Il faut contrôler les données reçues du front et s'assurer qu'elles soient conformes

3. Il faut définir des rôles pour contrôler qui peut recevoir quelle information

Périmètre non couvert / limites :

1. Les Endpoint POST peuvent avoir des failles

2. Les Endpoint DELETE peuvent avoir des failles

3. les vérifications de la base de données peuvent avoir des failles

6- Note d’auto-évaluation

Avons-nous considéré les spécicités mobile/IoT (token sur device, rmware, réseau non maîtrisé) ? ☐ Oui x Non

Chaque contre-mesure est-elle argumentée (et pas seulement listée) ? x Oui ☐ Non

Les compromis sont-ils explicites (sécurité vs. performance vs. UX) ? x Oui ☐ Non

Y a-t-il au moins une menace liée à ce que l'API retourne (et pas seulement ce qu'elle accepte) ? ☐ Oui X Non

Nos 3 risques prioritaires sont-ils justiés par des critères explicites ? X Oui ☐ Non